Affinity Propagation (AP)

Структурированный разбор метода Affinity Propagation с пояснением принципа работы, базовых математических соотношений, примером реализации на Python и задачами для проверки понимания.

Суть метода

Affinity Propagation - это алгоритм кластеризации, особенностью которого является отсутствие необходимости заранее задавать число кластеров. В отличие от методов, где требуется сразу указать количество групп, здесь сами объекты данных обмениваются числовыми сообщениями, на основании которых постепенно выделяются так называемые представители кластеров.

В качестве представителя кластера выступает одна из реальных точек выборки. Такая точка называется exemplar, то есть образец или экземпляр. Остальные точки привязываются к тем exemplar, которые оказываются для них наиболее подходящими.

Алгоритм работает итерационно. На каждой итерации пересчитываются два основных типа сообщений:

1. **Responsibility** - показывает, насколько хорошо точка-кандидат подходит на роль exemplar для другой точки.
2. **Availability** - показывает, насколько оправдано назначение данной точки exemplar для других точек.

Таким образом, метод не просто разбивает данные на группы, а сначала определяет, какие точки должны стать центрами кластеров, после чего формирует принадлежность остальных объектов к найденным представителям.

Основные понятия

1. Similarity

Сходство между двумя точками обычно задаётся через отрицательное евклидово расстояние:

$$s(i,k) = -||x_i - x_k||^2$$

Чем ближе точки друг к другу, тем выше значение сходства. Поскольку используется отрицательное расстояние, максимально похожими оказываются те точки, между которыми расстояние минимально.

2. Exemplar

Exemplar - это точка, которая представляет собой центр кластера не в абстрактном смысле, а как реально существующий объект из набора данных.

3. Responsibility

Responsibility $\mathbf{r}(\mathbf{i}, \mathbf{k})$ отражает, насколько точка k подходит быть exemplar для точки i. По сути, это оценка "насколько лучше выбрать именно этого кандидата, чем всех остальных".

4. Availability

Availability $\mathbf{a}(\mathbf{i}, \mathbf{k})$ отражает, насколько точка k вообще доступна для точки i в качестве exemplar, если учитывать мнение остальных точек.

Математическая основа метода

Обновление ответственности

$$r(i,k) \leftarrow s(i,k) - \max_{k' \neq k}(a(i,k') + s(i,k'))$$

Это выражение сравнивает кандидата k со всеми остальными возможными exemplar. Если точка k заметно лучше других кандидатов, её ответственность возрастает.

Обновление доступности

Для случая $\mathbf{i} = \mathbf{k}$:

$$a(i,k) = \min\left(0,\ r(k,k) + \sum_{i' \notin \{i,k\}} \max\left(0,\ r(i',k)\right)\right)$$

Это означает, что доступность формируется на основе того, насколько другие точки поддерживают выбор точки k в качестве exemplar.

Для диагональных элементов

$$a(k,k) = \sum_{i' \neq k} \max\left(0,\ r(i',k)\right)$$

Диагональный элемент отвечает за "самоподдержку" точки в роли exemplar.

Демпфирование

Чтобы избежать колебаний и нестабильности, в метод вводится демпфирование:

$$r_{t+1}(i,k) = \lambda \cdot r_t(i,k) + (1-\lambda) \cdot r_{new}(i,k)$$

$$a_{t+1}(i,k) = \lambda \cdot a_t(i,k) + (1-\lambda) \cdot a_{new}(i,k)$$

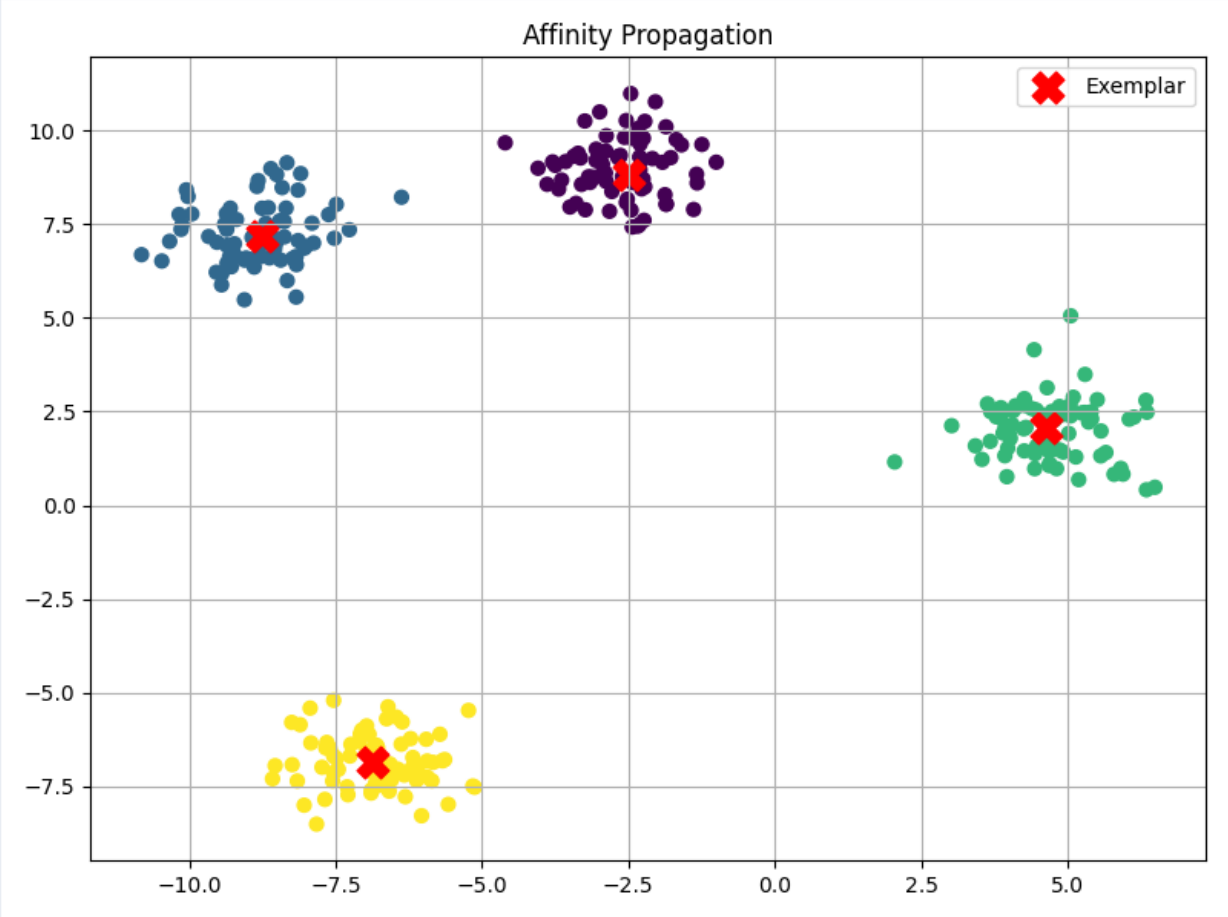
Здесь λ - коэффициент затухания, который обычно берётся в диапазоне от 0.5 до 1.

Реализация на Python с использованием scikit-learn

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import AffinityPropagation
from sklearn.datasets import make_blobs

# 1. Генерация данных
X, _ = make_blobs(
    n_samples=300,
    centers=4,
    cluster_std=0.8,
    random_state=42
)

# 2. Создание и обучение модели
model = AffinityPropagation(
    damping=0.85,
    preference=-60,
    random_state=42
)

model.fit(X)

# 3. Получение результатов
labels = model.labels_
cluster_centers = model.cluster_centers_indices_
n_clusters = len(cluster_centers)

print(f"Количество найденных кластеров: {n_clusters}")
print(f"Индексы exemplar: {cluster_centers}")
```

```
# 4. Визуализация
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', s=40)
plt.scatter(
    X[cluster_centers, 0],
    X[cluster_centers, 1],
    c='red',
    s=220,
    marker='X',
    label='Exemplar'
)
plt.title("Affinity Propagation")
plt.legend()
plt.grid(True)
plt.show()
```

```
Количество найденных кластеров: 4
Примеры индексов: [138 180 237 247]
```



Пояснение к примеру

В приведённом примере сначала создаётся искусственный набор данных с четырьмя группами точек. После этого инициализируется модель Affinity

Propagation. В параметре damping задаётся коэффициент сглаживания обновлений, а в preference - склонность точки становиться exemplar.

После обучения можно получить:

- метки кластеров для всех точек,
- индексы точек, выбранных в качестве exemplar,
- итоговое количество найденных кластеров.

На графике обычные точки окрашиваются по принадлежности к кластерам, а exemplar выделяются отдельными красными маркерами.

Основные параметры модели

Параметр	Описание	Влияние
damping	коэффициент затухания обновлений	влияет на устойчивость и сходимость
preference	склонность точки стать exemplar	чем выше значение, тем больше обычно получается кластеров
affinity	способ вычисления сходства	задаёт, как определяется близость между объектами
max_iter	максимальное число итераций	ограничивает время работы алгоритма
convergence_iter	число итераций для проверки сходимости	определяет, когда алгоритм считается стабилизировавшимся

Достоинства метода

1. Не нужно заранее задавать число кластеров.
2. Центры кластеров выбираются из реальных точек набора данных.
3. Отсутствует зависимость от случайного выбора стартовых центров, как в некоторых других алгоритмах.
4. Метод удобен в тех случаях, когда заранее неизвестно, сколько групп присутствует в данных.

Недостатки метода

1. Высокая вычислительная сложность.
2. Значительное потребление памяти.
3. Чувствительность к параметрам preference и damping.
4. При больших объёмах данных алгоритм может работать слишком долго.

Задачи для проверки понимания

Задача 1. Влияние preference

Вопрос: Как изменится количество кластеров, если увеличить значение preference?

Ответ:
Если увеличить preference, то каждая точка получает больше шансов стать exemplar. Вследствие этого число кластеров обычно возрастает. Если же сделать preference более отрицательным, то exemplar станет меньше, а значит уменьшится и количество кластеров.

Задача 2. Влияние damping

Вопрос: Для чего нужен параметр damping и что может произойти, если задать слишком маленькое значение?

Ответ:
Параметр damping нужен для сглаживания обновлений сообщений между точками. Это необходимо, чтобы итерационный процесс не начал колебаться. Если задать слишком маленькое значение, алгоритм может стать нестабильным и не прийти к сходимости за разумное число шагов.

Задача 3. Оценка сложности

Вопрос: Почему Affinity Propagation неудобен для очень больших наборов данных?

Ответ:
Причина состоит в том, что алгоритм использует матрицы сходства и сообщений между всеми парами точек. Это приводит к квадратичному росту памяти и большим временным затратам. При увеличении числа объектов вычисления становятся слишком тяжёлыми, особенно если данных десятки тысяч.

Задача 4. Практическая настройка параметра

Условие:
Пусть имеется набор данных из 500 точек, в котором предполагается около 5 реальных кластеров. Требуется подобрать значение preference так, чтобы получаемое число кластеров было как можно ближе к 5.

Решение:

```
import numpy as np
from sklearn.cluster import AffinityPropagation
from sklearn.datasets import make_blobs
```

```
from sklearn.metrics import silhouette_score

# Генерация данных
X, _ = make_blobs(
    n_samples=500,
    centers=5,
    cluster_std=0.75,
    random_state=42
)

best_pref = None
best_score = -1
best_n_clusters = None

for pref in np.linspace(-120, -10, 12):
    model = AffinityPropagation(
        damping=0.85,
        preference=pref,
        random_state=42
    )

    labels = model.fit_predict(X)
    n_clusters = len(np.unique(labels))

    if n_clusters > 1:
        score = silhouette_score(X, labels)

        if best_n_clusters is None:
            best_pref = pref
            best_score = score
            best_n_clusters = n_clusters
        else:
            current_diff = abs(n_clusters - 5)
            best_diff = abs(best_n_clusters - 5)

            if current_diff < best_diff or (current_diff == best_diff and score >
best_score):
                best_pref = pref
                best_score = score
                best_n_clusters = n_clusters

print(f"Лучший preference: {best_pref}")
print(f"Найдено кластеров: {best_n_clusters}")
print(f"Silhouette Score: {best_score:.3f}")
```

Пояснение:
В этом примере поочерёдно перебираются разные значения preference. Для каждого варианта вычисляется число кластеров и метрика качества Silhouette Score. Затем выбирается тот параметр, при котором найденное число кластеров оказывается ближе всего к ожидаемому значению, а среди равных вариантов предпочтение отдаётся более качественному разбиению.

Итог

Affinity Propagation представляет собой интересный и достаточно мощный метод кластеризации, который особенно полезен в тех ситуациях, когда число кластеров заранее неизвестно. Его сильной стороной является автоматический выбор exemplar и отказ от необходимости заранее задавать количество групп. При этом на практике метод требует внимательного подбора параметров и плохо масштабируется на очень большие выборки.